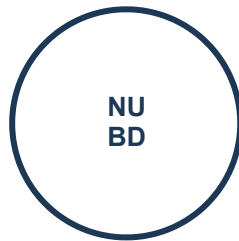


NATIONAL UNIVERSITY, BANGLADESH

SaaS-Based Human Resource Management System

A Multi-Company Recruitment and Employee Administration
Platform



A Project Thesis Submitted By

Md Abdullah

Registration No.: 20502002717

Session: 2021–2022

**BACHELOR OF SCIENCE
IN
COMPUTER SCIENCE AND ENGINEERING**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DHAKA CITY COLLEGE
Affiliated with National University, Bangladesh**

Declaration

I hereby declare that this undergraduate project thesis, entitled “SaaS-Based Human Resource Management System: A Multi-Company Recruitment and Employee Administration Platform,” is submitted to the Department of Computer Science and Engineering, Dhaka City College, affiliated with National University, Bangladesh, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

The software described in this report is based on the Laravel project available in the accompanying workspace at the time of preparation. Implemented functions are distinguished from proposed improvements. No unverified performance measurement or test outcome is presented as completed work. Sources and technical documentation used in preparing this report are acknowledged in the references.

I confirm that this report has not previously been submitted, in whole or in substantial part, to another institution for an academic qualification. The final submission remains subject to review, correction, and approval by my project supervisor and the Department.

Signature: _____ Date: _____

Md Abdullah | Registration No. 20502002717 | Session 2021–2022

Dedication

THIS PROJECT IS DEDICATED TO MY BELOVED PARENTS AND HONOURABLE TEACHERS.

With gratitude for their patience, encouragement, and support throughout my studies.

Certificate of Approval

The project thesis entitled “SaaS-Based Human Resource Management System: A Multi-Company Recruitment and Employee Administration Platform,” submitted by Md Abdullah, Registration No. 20502002717, Session 2021–2022, is presented to the Department of Computer Science and Engineering, Dhaka City College, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

The date of acceptance and examiner details are intentionally left for completion through the official departmental process.

Supervisor: Md Shahiduzzaman, Assistant Professor, Department of CSE, Dhaka City College.

Department Head: Md Jahangir Alam, Associate Professor and Head.

External Examiner 1: Name _____ Designation _____ Institute _____

External Examiner 2: Name _____ Designation _____ Institute _____

Approval date: _____

Acknowledgment

I express my sincere gratitude to Almighty Allah for the strength, health, and perseverance that made it possible to complete this project and prepare this report.

I am deeply grateful to my project supervisor, Md Shahiduzzaman, Assistant Professor, Department of Computer Science and Engineering, Dhaka City College, for guidance, constructive feedback, and encouragement during the development of this project. His supervision helped me connect software engineering concepts with the practical requirements of human resource administration.

I would also like to thank Md Jahangir Alam, Associate Professor and Head of the Department, and all faculty members for their academic support. I appreciate the maintainers of Laravel, Vue.js, Inertia.js, Tailwind CSS, and the open-source packages used in the application. Finally, I thank my parents, family, classmates, and friends for their patience and support. Any remaining errors are my responsibility.

Abstract

Human resource teams in small and medium-sized organizations often coordinate recruitment through email, spreadsheets, paper documents, and informal messaging. This fragmentation can make it difficult to maintain a consistent application process, organize candidate assessments, protect documents, and move a selected applicant into an employee record.

This project presents a web-based, multi-company Human Resource Management System focused on recruitment and basic employee administration. It is implemented with Laravel 12, PHP, a relational database accessed through Eloquent ORM, Vue.js, Inertia.js, and Tailwind CSS. A company tenant can manage job positions and assessment questions, share a public application link, collect candidate details, and guide applicants through staged assessment and document-submission screens. Administrators can review applications, update status, approve a candidate to create an employee account, search employee records, view profiles, and remove employee accounts.

The report documents the architecture, data model, workflows, access model, verification strategy, limitations, and future work. It does not claim payroll, attendance, leave management, or measured production performance, because those functions and measurements are not evidenced in the inspected project.

Keywords: Human Resource Management, SaaS, multi-tenancy, recruitment, candidate assessment, Laravel, Vue.js, employee administration.

Contents

Front matter: Declaration; Dedication; Certificate of Approval; Acknowledgment; Abstract; Contents; Lists of Tables and Figures; Abbreviations.

Chapter 1: Introduction. Background, motivation, problem statement, objectives, scope, methodology, and summary.

Chapter 2: Related Work and Domain Study. HR information systems, recruitment lifecycle, multi-tenancy, data protection, comparative analysis, and research gap.

Chapter 3: Requirements and Feasibility. Development method, feasibility, functional and non-functional requirements, actors, use cases, and acceptance criteria.

Chapter 4: System Design and Analysis. Architecture, stack, deployment, tenant isolation, schema, data dictionary, candidate lifecycle, access control, and security.

Chapter 5: Implementation and Verification. Core models and modules, routes, file handling, verification strategy, test matrix, current test evidence, and limitations.

Chapter 6: Conclusion and Future Work. Contributions, limitations, and development roadmap. References and four appendices follow.

List of Tables and Figures

Tables: Functional requirements; non-functional quality objectives; actor and permission overview; technology stack; entity relationship summary; company, user, candidate, question, and assessment data dictionaries; candidate status transitions; security risks and mitigations; route summary; verification cases; and roadmap priorities.

Figures: System context; high-level architecture; multi-company data boundary; candidate funnel; candidate status lifecycle; entity relationship diagram; authorization request pipeline; employee conversion sequence; and proposed module roadmap.

The diagrams in this report are analytical schematics derived from the application structure. They are not screenshots or evidence of additional runtime features.

List of Abbreviations

CRUD: Create, Read, Update, Delete. CSRF: Cross-Site Request Forgery. DBMS: Database Management System. ERD: Entity Relationship Diagram. HR: Human Resources. HRM: Human Resource Management. HTTP: Hypertext Transfer Protocol.

MVC: Model-View-Controller. ORM: Object-Relational Mapping. RBAC: Role-Based Access Control. SaaS: Software as a Service. SPA: Single-Page Application. SQL: Structured Query Language. UI: User Interface. UAT: User Acceptance Testing. UX: User Experience.

In this report, tenant means an organization whose records are logically separated by a company identifier. Candidate means a person who started or submitted a job application. Employee means an authenticated user associated with a company after administrator approval.

Introduction: Background

Human resource administration connects an organization's workforce needs with people who apply to meet them. A hiring cycle involves defining a vacancy, communicating with applicants, collecting personal information, screening, reviewing documents, making decisions, and onboarding selected candidates. When each stage uses separate spreadsheets, messages, and paper folders, information can be duplicated, status becomes unclear, and access to sensitive records is hard to control.

Web-based HR systems centralize records and provide consistent processes. A multi-company service adds the requirement that each organization work within its own logical data boundary while a platform operator administers tenant accounts. The application studied in this thesis addresses that setting through a company-oriented recruitment workflow and a basic employee directory.

The project workspace contains Laravel 12, PHP, Inertia.js, Vue.js, and Tailwind CSS. Its entities include companies, users, candidates, positions, questions, and candidate assessments. Public routes expose company-specific application steps; authenticated routes provide company administration and employee management.

System boundary: attendance, payroll, leave approval, performance appraisal, and benefits administration are not implemented in the inspected application.

Motivation and Problem Statement

Small organizations may not have a dedicated applicant tracking platform. Vacancy responses arrive through multiple channels and hiring staff manually compare candidates, request documents, and track interview or hold decisions. This is workable at low volume but becomes error-prone as candidate numbers and job roles increase.

The project is motivated by three needs: applicants need a clear sequence of steps; administrators need candidate contact data, responses, files, and status in one review area; and a software service for multiple organizations must associate each record with the correct company and restrict actions by role.

The central problem is the absence of one traceable workflow linking vacancy definition, applicant registration, assessment, document collection, application review, selection, and employee account creation. Fragmentation can introduce incomplete records, inconsistent questions, delayed decisions, and accidental exposure of candidate information.

- Stakeholders include platform administrators, company administrators, applicants, and employees.

Objectives and Research Questions

General objective: design, implement, and document a web-based multi-company HR application that supports a structured recruitment process and essential employee administration.

Specific objectives are to model company-scoped data; provide company position and question administration; implement candidate registration, assessment, document submission, and confirmation; support application review and employee conversion; apply authentication, roles, validation, and tenant identifiers; and assess the system using a practical verification plan.

Research questions: How can one application organize recruitment data for multiple companies? How can a guided funnel reduce fragmented data collection? Which controls protect company and candidate data? What further verification is required before production use?

These questions are examined through source inspection, requirements analysis, schema review, workflow modeling, and test-case design. They do not constitute a controlled user study or performance benchmark.

Scope and Boundaries

Included functions are company records; user authentication and role assignment; position management; question configuration; public candidate registration; IQ and position-specific assessments; resume, identity, and certificate upload paths; candidate review and status changes; approval into an employee account; employee search, profile view, and account removal.

The project does not provide a verified attendance ledger, payroll, leave requests, performance appraisal, recruitment email service, subscription billing, complete audit logging, or production monitoring dashboard. A check-in button and sample dashboard metrics are not evidence of functioning modules.

The report assumes the institution and session in the supplied thesis sample: Dhaka City College, affiliated with National University, Bangladesh, Session 2021–2022. The department head is retained from that sample; student and supervisor details follow the user’s instruction. Approval date and external examiner fields remain blank.

The supervisor and department should confirm the institutional details, title, and format before binding.

Methodology and Contributions

The project follows an iterative software engineering approach. The domain was narrowed to recruitment and employee administration. Routes, controllers, Vue pages, migrations, models, and seeders were reviewed to establish the implemented feature set. Requirements were grouped into user workflows and quality concerns, and relationships and access boundaries were analyzed before documenting the design.

Implementation uses Laravel MVC, Inertia page responses, Vue components, and Eloquent models. Verification should combine route-level HTTP tests, database tests, browser workflow tests, and security checks. The repository contains only a basic example feature test; this report therefore specifies a verification plan rather than claiming comprehensive executed results.

Contributions include a workflow linking vacancy configuration with applicant review; a company-scoped data model; configurable question formats; a candidate funnel with document submission; and a concrete analysis of tenant isolation, privacy, and lifecycle risks.

Chapter Summary

This chapter framed the project as a SaaS-oriented recruitment and employee administration application. The problem is fragmented vacancy and applicant workflows, inconsistent records, and the need to separate organizations' data. The implementation centralizes company, position, assessment, candidate, and employee records in a Laravel application.

The objectives emphasize structured candidate intake, configurable vacancies and questions, company-scoped administration, employee conversion, and a security review. Attendance, payroll, leave, and performance are explicitly outside the implementation boundary.

Evidence in this report comes from the source tree and migrations. Gaps are recorded where code indicates incomplete validation or where automated tests cannot support a broad quality claim. Chapter 2 reviews the HR system domain and the engineering gap addressed by this project.

Related Work: HR Information Systems

A Human Resource Information System organizes workforce information and supports administrative processes. A small recruitment tracker may manage applicants and vacancies, while an enterprise platform can include payroll, attendance, learning, benefits, and workforce planning. Appropriate scope depends on organizational needs, legal context, and available resources.

Recruitment software commonly supports vacancy definition, application capture, candidate records, screening, interview coordination, decision tracking, and conversion to an employee profile. Effective systems link each applicant to the relevant position, allow authorized reviewers to inspect information, and preserve clear status.

This project implements several recruitment stages but not a complete HR lifecycle suite. Its contribution should be assessed as a focused prototype and a foundation for extension, not as parity with enterprise products.

Recruitment Lifecycle and Process Risks

A hiring lifecycle begins when an organization defines and publishes a vacancy. A candidate provides identity and contact data, selects a position, submits screening responses, and supplies documents. HR staff review the application, update a decision, and either close the application or approve the candidate. Approval should create an employee record without duplicating identity data or exposing credentials.

Manual processes create inconsistent status labels, incomplete assessment records, and unclear document ownership. Attachments sent by email may be stored outside the candidate record. A multi-company service adds a risk that a missing company filter exposes records to another tenant.

A guided sequence can clarify applicant expectations and store responses near the candidate record. Server persistence is authoritative; browser local storage may preserve unfinished responses but should be treated cautiously on shared devices. The inspected assessment page uses local storage for progress and posts answers to the server on phase submission.

Multi-Tenant Design

A multi-tenant application hosts multiple organizations in one deployment. Separation can use individual databases, schemas, or shared tables with a tenant key. Shared tables are economical but make application scoping and database constraints especially important.

This project uses shared tables and company identifiers to associate companies, users, candidates, positions, and questions. A regular administrator's effective company context should come from the authenticated user. A super administrator can deliberately select a tenant workspace using session state. Every lookup and mutation still requires ownership checks.

Isolation principles are: derive tenant context from trusted server state; scope reads, writes, deletions, and downloads; reject client-supplied tenant identifiers; reinforce rules with constraints; test cross-tenant access; and audit privileged tenant switching. Centralized policy or tenant-context services can reduce controller inconsistency.

Privacy and Candidate Data Protection

Recruitment data includes names, email addresses, telephone numbers, assessment responses, resumes, identity documents, and certificates. It warrants restricted access, clear purpose, secure transmission, retention limits, and deletion procedures. This chapter describes engineering safeguards and is not legal advice; operators must determine applicable privacy obligations.

Security objectives include confidentiality, integrity, availability, and accountability. Laravel provides useful mechanisms such as session authentication, CSRF protection, validation, password hashing, and authorization. Correct application configuration remains essential.

The candidate pages render question and answer content with Vue v-html. This should be limited to trusted, sanitized content or replaced with escaped text. Uploaded identity files should be stored privately and delivered through authorization-checked downloads rather than predictable public links.

Observed concern: HTML rendering of question text can introduce cross-site scripting risk unless content is trusted and sanitized.

Comparative Analysis

Spreadsheet processes provide flexible lists but weak workflow enforcement and access control. Applicant tracking products commonly provide vacancy publishing, candidate pipelines, and evaluations. HRIS suites often add employee records and other workforce functions. A multi-tenant SaaS must enforce company separation in addition to these workflows.

The project includes company-specific position configuration, a public application route, staged candidate intake, question management, application review, and account conversion. It does not implement payroll or attendance. This qualitative comparison describes broad categories and is not a benchmark of named commercial products.

A rigorous commercial comparison would require a documented feature set, trial environment, consistent criteria, and evidence for every rating. This report positions the project as a focused recruitment workflow between manual files and a complete HR suite.

Research Gap and Summary

The domain gap lies between isolated candidate forms and a complete HR information system. A small organization needs vacancy management, consistent intake, configurable screening, document review, decisions, and transition into an employee record. Multi-organization hosting additionally requires logical separation and privacy controls.

The application addresses the recruitment portion through company-associated applications, assessments, file paths, review, status, and employee records. It lacks comprehensive audit history, verified private file access, and sufficient automated tests.

Precise scope is essential: a check-in mockup is not an attendance service, and an optional score column is not a scoring implementation. Chapter 3 converts these findings into requirements and acceptance conditions.

Development Method and Feasibility

An iterative method suits this project because the workflow divides into reviewable increments. A reasonable sequence is authentication and tenants; positions and questions; public registration and assessment; documents and candidate review; employee conversion; then regression and security testing.

Technical feasibility is supported by Laravel, PHP, Vue, Inertia, and a relational database. Operational feasibility depends on browser access, reliable hosting, and user guidance. Open-source components reduce licensing costs but do not remove hosting, backup, security-review, and support costs. Schedule feasibility depends on keeping payroll and attendance out of the initial scope.

The sequence is a recommended development method, not a verified project schedule. A maintained issue log and release checklist would improve traceability.

Functional Requirements I

FR-01: authenticate administrative users and provide login, logout, password reset, and email verification. FR-02: allow platform administration to create, edit, list, and delete company records. FR-03: allow a privileged operator to enter and exit a company workspace.

FR-04: allow company users to create positions and manage active status. FR-05: allow question configuration by company. FR-06: provide a public company-specific registration page with active position choices. FR-07: collect candidate IQ and position-specific departmental answers.

These requirements describe capabilities found in routes and components. Presence in code does not prove correct authorization or error behavior; each needs a test for both allowed and denied operations.

- Requirement priority should be reviewed with the supervisor and intended users.

Functional Requirements II

FR-08: accept resume, identity, and certificate documents with server validation. FR-09: display company candidates and support review and status changes. FR-10: approve a candidate and provision an employee account. FR-11: search employee records, view profile information, and remove an employee account.

Candidate status updates and account conversion are high-impact actions. They should validate company ownership, permitted state transitions, and the actor's authority. Approval should be safe when retried and should not expose credentials.

A route-level traceability matrix should connect each requirement to controllers, models, Vue pages, migrations, and tests. This makes it easier to detect a screen that is present without a functioning server-side workflow.

Non-Functional Requirements

Security requires each protected action to enforce authentication, role, and tenant context. Privacy requires private documents, controlled downloads, and retention rules. Integrity requires valid relationships and consistent application status. Usability requires clear progress and recoverable errors. Maintainability requires clear responsibility boundaries. Availability requires backups and recovery procedures.

Performance targets must be agreed with the operator and measured against a defined dataset and environment. No latency or concurrency target was measured while preparing this report, so no numerical response-time claim is made.

Constraints include session-based candidate context, filesystem and web-server upload limits, shared question formats between Laravel and Vue, consistent seeded role names, and secure production configuration for HTTPS, application secrets, mail, storage, and backups.

Actors and Use Cases

Super Admin manages tenant records and may enter or exit a selected company workspace. Company Admin manages positions, questions, applications, and employee accounts in the assigned tenant. Candidate registers publicly, completes assessments, and submits documents. Employee signs in to the authenticated application view; current employee functions remain limited.

Vacancy use case: administrator adds a title, system associates it with the tenant, administrator activates it, and the public page lists it for applicants. Application review use case: administrator selects a company-scoped candidate, reviews answers and documents, updates status or approves, and should leave an attributable decision record.

External integrations such as mail delivery, payroll, biometrics, and payment gateways are not evidenced in the repository. They should be added only after their data exchange and security model are specified.

Acceptance Criteria and Summary

A valid candidate can register for an active position; assessment answers persist across phases; unsupported or oversized files are rejected; administrators see only their tenant's records; repeated approval does not duplicate employee accounts; unauthorized roles cannot perform company administration; and deletion follows a documented retention policy.

These criteria should become executable tests using a dedicated test database, synthetic people, fake filesystem storage, and fake mail transport. Cross-tenant tests should verify both denial and unchanged database state.

Chapter 3 established the requirement baseline. Production readiness also depends on authorization audits, private file access, migration review, tests, backups, and credential management. Chapter 4 documents the architecture and data design.

Architecture Overview

The application follows a server-rendered web architecture. Laravel handles routing, authentication, validation, controllers, database operations, and page responses. Inertia connects Laravel routes to Vue components without a separately deployed API service. Tailwind CSS provides utility styling and Eloquent maps relational records to PHP models.

A protected request passes through authentication and verification middleware where configured. A controller validates input, reads or updates models, and returns an Inertia page or redirect. Vue forms submit through Inertia and display server validation errors. Candidate documents use the configured Laravel filesystem.

The application is a monolith, appropriate for a focused student project. As the system grows, extracting reusable services and authorization policies can make controllers more testable and reduce duplicated tenant checks.

- Browser → Laravel routes/middleware → controllers → Eloquent → relational database. Vue pages ↔ Inertia responses. Uploaded files → configured storage disk.

Technology Stack

Composer declares PHP 8.2 or later, Laravel 12, Inertia Laravel, Sanctum, and Spatie Permission among dependencies. The frontend manifest includes Vue 3, Inertia Vue, Vite, Tailwind-related packages, Axios, and supporting utilities. Vite builds browser assets; Laravel serves routes and page data.

The project uses a JavaScript build tool but is not a MERN system: server-side application logic is Laravel/PHP and the front end uses Vue. Exact production versions should be taken from lock files and deployment configuration.

Keeping the application in one codebase reduces integration complexity. Inertia preserves Laravel routing and validation conventions while enabling reactive Vue interfaces. Eloquent simplifies relational CRUD; controller business logic may need service extraction as workflows become more complex.

Deployment Topology

Production requires a PHP-compatible web server, Laravel application, relational database, persistent document storage, and compiled frontend assets. HTTPS protects credentials, session cookies, and candidate information in transit. Secrets belong in protected deployment configuration and must not be committed.

Readiness work includes disabling debug mode, using a unique app key, enforcing secure cookies, least-privilege database credentials, private document storage, tested mail delivery, encrypted backups, restore exercises, monitoring, and release/rollback procedures.

These are recommendations, not assertions about the current hosting environment. The repository does not evidence an audited production topology or measured availability.

- Browser $\rightleftharpoons$ HTTPS web server/PHP $\rightleftharpoons$ Laravel application $\rightleftharpoons$ database; Laravel $\rightleftharpoons$ private storage; scheduled backups $\rightleftharpoons$ protected backup target.

Company Data Isolation

Companies are the tenant roots. A company owns positions, questions, candidates, and users; an assessment belongs to a candidate and inherits the candidate's tenant through that relation. The schema can represent tenant ownership, but controllers must enforce it on every read and mutation.

For regular accounts, company identity should come from authenticated server state, not an untrusted request. A super administrator may select a company workspace, but the server must check the role and consistently resolve that context. Each handler must validate its tenant scope.

Candidate and employee lists show company-scoping patterns, including active company selection for a super administrator. Other controllers should be reviewed individually; some use only the signed-in user's company and may not honor impersonated context consistently.

A list-page filter alone cannot guarantee isolation for detail, update, approval, deletion, or document access.

Database Model

Primary tables are companies, users, positions, questions, candidates, and candidate_assessments. Companies relate to users, positions, questions, and candidates. Candidate assessments reference candidate records and cascade on candidate deletion. Spatie Permission creates role and permission tables and polymorphic pivots.

Candidate-to-employee linkage is currently indirect: approval matches by email instead of storing an explicit user foreign key on the candidate. Position is stored as text on candidate rather than a foreign key. These choices can be workable in a prototype but weaken referential integrity when emails or titles change.

A normalized extension could store a stable position identifier plus a title snapshot and a direct candidate-to-user link. Transactions should protect company onboarding and candidate approval from partial failure.

- companies 1—* users; companies 1—* positions/questions/candidates; candidates 1—0..1 assessments; approval link is currently indirect.

Data Dictionary: Company and User

Company fields include primary key, name, unique email, optional phone and domain, status (active, inactive, suspended), and timestamps. The domain field is described as future custom-domain support; custom-domain routing is not evidenced.

User fields include nullable `company_id`, name, unique email, optional phone, email verification timestamp, password hash, `is_active` flag, remember token, and timestamps. Platform accounts may have no company. User model hides password and remember token from serialization and casts password to a hash.

Company deletion removes associated users explicitly and relies on configured foreign-key behavior for company children. Destructive behavior should be compared with retention obligations. Soft deletion or tenant suspension may be safer where records must be retained.

Data Dictionary: Candidate and Position

Candidate fields include `company_id`, `name`, `email`, `phone`, selected position text, optional IQ and department score integers, `resume/NID/certificate` paths, free-form status defaulting to Pending, and timestamps. The migration does not impose company-scoped email uniqueness.

Positions include `company_id`, `title`, optional JSON questions field, active flag, and timestamps. The candidate position title is not linked by foreign key. A stable identifier improves integrity; a title snapshot preserves historical context when the job title changes.

Score columns do not establish that a calculation exists. The inspected answer persistence helper does not calculate the score. A scoring implementation would require explicit answer keys, score rules, server-side evaluation, and tests.

Data Dictionary: Questions and Assessments

Questions include optional `company_id`, question text, type, options JSON, category, optional department, and timestamps. Null `company_id` can represent global templates. The interface recognizes multiple choice, yes/no, and written response types; values should be constrained server-side.

Candidate assessments include `candidate_id`, JSON answers, optional integer score, and timestamps. Model casts options as arrays, while assessment helper explicitly encodes/decodes answer JSON. Consistent JSON casting and payload validation would reduce type ambiguity.

Assessment history should preserve the exact question wording and choices shown at response time. Versioning avoids historical answers becoming uninterpretable after administrators edit or delete a question.

Candidate Application Workflow

A company-specific route resolves the company name and lists active position titles. Registration collects name, email, phone, and position. A candidate identifier and company identifier are stored in session for later stages.

The IQ phase combines global questions and matching company questions for the selected position or an All category. The departmental phase retrieves company questions matching the selected position. Answer submissions merge into a JSON assessment record.

The document phase validates file extensions and size, writes to the configured public disk, and saves paths. Rules screen follows, then a confirmation screen reads the candidate from session. Direct navigation, session expiry, shared devices, and cross-company session changes need explicit handling.

- Company route → registration → IQ assessment → departmental assessment → documents → rules → confirmation → administrator review.

Candidate Status Lifecycle

The candidate migration stores status as an unconstrained string. Observed values include Applied, Pending, Call for Interview, Hold, Rejected, and approved/Approved in different code and views. Approval writes lowercase approved while the interface checks uppercase Approved, so badges and action visibility can disagree.

A robust state model defines canonical values, allowed transitions, actor guards, and an audit record. The client may show friendly labels while the server persists stable identifiers. Approval must be idempotent and must not disclose a default password.

Recommended transitions include Applied to Interview, Hold, Rejected, or Approved. Every transition should confirm tenant ownership and record actor, timestamp, and reason. The system should define whether rejected candidates can be reopened and how long decisions remain available.

Role-Based Access Control

The project uses Spatie Laravel Permission and seeds Super Admin, Company Admin, Employee, and Candidate roles. User model includes the role trait. Controllers check platform role for company switching, and dashboard pages use roles to select presentation.

Company routes are grouped behind authentication and email verification, but explicit role middleware is not uniformly visible in the route file. UI hiding does not protect server operations. Policies or role middleware should enforce capabilities and tests should cover every route.

Candidate funnel routes are session-based and public, even though a Candidate role exists. A role record should not be assumed to secure public application data. Candidate-specific status access requires a safe identity or token design.

Authentication and Request Security

Authentication routes support login, logout, password reset, password confirmation, and email verification. Protected routes use `auth` and `verified` middleware in the main route file. Authorization must additionally check role, company ownership, request validation, and safe response serialization.

Company creation sets a predictable initial password; candidate approval derives a default password from the company name and returns it in a flash message. These patterns are unsafe for production. Use one-time expiring invitations, require a new password, and never disclose reusable credentials.

Rate-limit login and password reset, enforce session invalidation, disable inactive accounts during authentication, and audit tenant switches, approvals, status changes, and destructive operations. Privileged multi-factor authentication may be appropriate.

- Authentication answers who is signed in; authorization answers what that user can do in the current tenant.

File Handling and Input Safety

The document handler accepts resumes as PDF/DOC/DOCX and identity/certificate files as PDF or common images, with a five-megabyte limit. The interface marks resume and identity required, but server rules are nullable. User-visible required state and server behavior therefore differ.

Files are stored on a public disk and candidate pages create direct /storage links. This can expose sensitive material to anyone with a URL, depending on deployment. Use private storage, authorization-checked downloads, random object names, size controls, and a defined retention policy.

Validate content type as well as extension. Restrict assessment keys and answer shapes; constrain question type and category; sanitize HTML; rate-limit public submissions; and avoid trusting client-side validation. Consent, if required, needs server-side persistence with version and timestamp.

Interface Design and Summary

Candidate pages provide registration, question controls, document input, rules acknowledgement, and a confirmation view. Administrative screens provide company forms, position/question management, candidate filtering and detail, and employee search/profile. Employee search is debounced; candidate filters are performed in the browser over records delivered by the server.

Usability work should clarify current phase and required information, place validation messages near fields, support keyboard and screen-reader interaction, confirm destructive actions, paginate larger lists, and avoid unnecessary exposure of personal data in browser storage.

Chapter 4 described a Laravel monolith with shared relational tables and company identifiers. Security relies on role authorization, tenant scoping, private files, safe rendering, and credential hygiene, not authentication alone. Formal accessibility and performance evaluations have not been performed.

Implementation Environment and Models

Composer declares PHP 8.2+, Laravel 12, Inertia Laravel, Sanctum, and Spatie Permission. Frontend dependencies include Vue 3, Inertia Vue, Vite, Tailwind-related packages, and supporting utilities. The repository uses Laravel conventions for controllers, models, migrations, seeders, routes, and Vue pages.

Core models are Company, User, Candidate, Position, Question, and CandidateAssessment. Company exposes users, questions, and candidates. Candidate belongs to company and has one assessment. Position belongs to company and casts JSON. Question belongs to company and offers global IQ templates.

Candidate-to-user and candidate-to-position relationships should be made explicit as the system matures. The current implementation maps several features through copied labels or email lookup.

Company Administration

Company administration lists, creates, edits, updates, and deletes tenant records. Creation validates company name/email, creates a company, creates an initial user, and assigns Company Admin. Super Admin can enter and exit a company context held in session.

Onboarding should be transactional to avoid partial records. Initial credentials must use an expiring activation flow rather than a predictable password. Deletion should comply with retention requirements. Tenant switching should be authorized, consistently honored by all controllers, and audited.

Routes use authentication and email verification, but role enforcement should be explicit through policies or middleware. Tests must show that non-admins cannot manage companies and that a company admin cannot operate on a different tenant.

Position and Question Management

Position administration creates titles and toggles active status. Public registration receives active titles for the selected company. The question builder supports multiple choice, yes/no, and written responses, with JSON options and company/department categories.

Questions should be validated against an allowlist, sanitized before HTML rendering, and versioned so historic answers remain interpretable. Department selection should use stable identifiers rather than free-form title matching. Scoring keys should not be sent to a public browser if scoring is introduced.

The position table contains a JSON questions field, while the builder primarily uses the questions table. The intended source of truth should be documented to avoid duplicate configuration paths.

Candidate Registration and Assessment

Registration resolves the company from a normalized URL slug, stores tenant state in session, and lists active position titles. The handler validates contact fields and position presence, checks for an existing email within the company, then creates a candidate with initial Applied status.

IQ questions combine global templates and matching company questions; departmental questions match company and position. Answers merge into candidate assessment JSON. A session stores candidate context between phases.

Position validation should confirm an active position belongs to the same company. Answers should be validated against known question identifiers and expected types. Duplicate email behavior should explicitly resume or safely report an existing application. The inspected helper does not calculate assessment scores.

Documents and Rules Phase

The document phase accepts a resume, identity document, and educational certificate. The handler applies extension and size rules, stores files on the public disk, and updates candidate paths. Administrative screens create direct storage links.

The rules screen includes an agreement checkbox, but its server handler redirects without validating or persisting that agreement. The UI indicates acceptance, yet the backend does not provide evidence of a durable consent record.

Before production, reconcile requiredness between UI and server, move sensitive files to private storage, add authorized downloads, persist terms version and acceptance time if required, and define retention and deletion. The checkbox alone is not proof of recorded consent.

Candidate Review and Decisions

The candidate page provides browser-side filters for name, email, phone, position, status, and date. A detail modal displays assessment responses and document links. Administrators can choose interview, hold, rejection, or approval.

Browser filtering is suitable only for small record sets and sends all supplied records to the client. Larger tenants need server-side pagination and search. Candidate detail and file access must independently verify tenant ownership; hidden buttons are not a security boundary.

Normalize status values, define allowed transitions, and record actor, old state, new state, reason, and timestamp. Keep question answer review separate from any answer key. Render only trusted sanitized HTML.

Employee Conversion and Administration

Candidate approval finds or creates a user by candidate email, copies name, phone, and company, assigns an employee role value, and updates candidate status. Employee pages list company users, support search, show a profile, and delete a user account to terminate access.

Approval should be transactional and idempotent. A direct candidate-user relationship is safer than mutable email matching. Spatie role assignment and any separate role string should not diverge. Onboarding should use a one-time link, not a predictable password. Deactivation may preserve history better than deletion.

Profile lookup by email associates employee to candidate data. Email changes can break that relation. A stable foreign key and explicit employment lifecycle would improve accuracy and auditability.

Route Summary

Authentication and profile routes support login, password reset, verification, and account management. Company routes cover list/create/edit/delete and tenant context. Protected recruitment routes cover applications, status update, approval, positions, questions, and employees.

Public funnel routes cover company registration, IQ, departmental assessment, documents, rules, and success. The inspected system uses browser web routes and session state, not a documented stateless JSON API. These routes should not be described as REST API endpoints.

A release-specific route inventory can be generated with Laravel `route:list`. Every protected handler should have a documented authorization policy, validation rule set, tenant ownership check, and test.

Verification Strategy

Verification should combine unit tests for small rules, feature tests for routes and database effects, browser tests for applicant and administrator flows, security tests for cross-tenant access and repeated approval, and manual accessibility evaluation.

Use a test database, fake filesystem, fake mail transport, and synthetic candidate data. Each bug should be recorded with reproduction, expected and actual behavior, and resolution. Test both permitted and nearby forbidden operations.

Performance testing requires a defined dataset, environment, workload, and repeatable measurements. No performance benchmark is included in this report.

Functional Test Matrix

TC-01 valid candidate registration creates a candidate under the correct company. TC-02 invalid email or missing data returns validation error without an incomplete record. TC-03 IQ and departmental answers persist without overwriting. TC-04 supported upload is stored and available only to an authorized reviewer.

TC-05 unsupported or oversized files are rejected. TC-06 a company administrator searches only own-tenant candidates. TC-07 status changes use valid states and record an actor. TC-08 repeated approval creates exactly one employee and no credential disclosure.

TC-09 Company A cannot retrieve Company B candidate data. TC-10 unauthenticated visitors cannot access protected pages. These are planned test cases; no pass status is claimed without executed evidence.

Execution state for each case: Not evidenced in the inspected repository.

Current Test Evidence

The inspected repository contains a basic example feature test asserting the home route returns HTTP 200. It does not establish recruitment workflow correctness, document handling, tenant isolation, role authorization, employee conversion, or assessment persistence.

No dedicated cross-company, upload, role, or performance tests were located during report preparation. Therefore, no pass rate, measured latency, or concurrency claim is made. The absence of a test is not the same as a failing test; it is an evidence gap.

A reproducible report should record commit identifier, environment, command, date, test counts, failures, and artifacts. Run the full suite and add critical workflow/security tests before final academic submission.

Do not fill execution records with estimated values.

Implementation Summary

The application provides a base for company management, vacancy and question configuration, candidate registration and assessment, document upload, application review, candidate decisions, employee account creation, and employee listing.

Important gaps include non-uniform tenant scoping evidence, public-disk document links, an unpersisted rules acknowledgement, predictable initial credentials, inconsistent status casing, score fields without a visible scoring calculation, and minimal automated tests.

These findings define follow-up work rather than invalidate the educational contribution. A trustworthy HR service treats authorization, privacy, and workflow integrity as implementation requirements. Chapter 6 summarizes contributions and proposes priorities.

Conclusion and Contributions

This project designed and implemented a web-based recruitment and employee administration workflow for multiple companies. It connects company and position setup to candidate intake, assessments, documents, review, decisions, and employee account creation. Laravel and Vue provide a practical base for future extensions.

The work demonstrates how a guided funnel can replace parts of an ad hoc process with associated records. Company identifiers provide a basis for separating tenant data and roles distinguish platform and company administration. The implementation remains a focused subset of HRM, not a full employee lifecycle suite.

A central conclusion is that feature completeness and operational readiness differ. Routes and pages do not prove secure access, durable consent, private files, safe account activation, or verified quality. Those properties require hardening and tests.

System Limitations

Functional limitations: no verified payroll, attendance, leave, performance, training, benefits, interview calendar, or communication automation. Dashboard controls and sample values must not be treated as operational features until connected to persisted workflows.

Data limitations: candidate status is free-form, position is stored as text, employee linkage uses email, response payload validation is incomplete, score columns do not evidence scoring, and rules agreement is not persisted.

Security/operations limitations: tenant isolation is not proven at every endpoint; files use public storage; initial credentials are predictable; HTML question content needs sanitization; automated tests are minimal; no backup, retention, or load-test evidence is included.

Prioritize credential safety, tenant isolation, document privacy, server validation, and automated security tests before collecting real candidate data.

Future Development Roadmap

Priority 1: centralized tenant policies with cross-company feature tests. Priority 2: private storage and authorized file downloads. Priority 3: secure, expiring employee invitations. Priority 4: canonical status enum and audit history.

Priority 5: question and answer validation plus safe rendering. Priority 6: position and candidate-user foreign keys with transactional approval. Priority 7: server-side pagination and accessibility evaluation. Priority 8: attendance, leave, or payroll modules only after domain and compliance review.

Future modules should share a stable employee identity model but remain independently permissioned and tested. Payroll and identity integrations have high correctness and privacy impact and require explicit calculation rules and domain-expert review.

Closing Remarks

The project illustrates that the difficult parts of a multi-company HR application extend beyond forms. The system must preserve the relationship between a tenant, vacancy, applicant, submitted documents, decision, and resulting employee account. Small inconsistencies in status, tenant context, or identity matching can undermine an apparently functional page.

Inspecting implementation evidence before making claims produces a more useful academic account. This report distinguishes implemented functions from proposed extensions and records gaps rather than presenting UI placeholders as completed services.

With tenant authorization, private document access, secure onboarding, input validation, consistent state transitions, and comprehensive tests, the system can develop from an academic prototype into a safer foundation for focused HR administration.

Academic and Technical Sources

Armstrong, M., and Taylor, S. *Armstrong's Handbook of Human Resource Management Practice*. 15th ed., Kogan Page, 2020.

Laudon, K. C., and Laudon, J. P. *Management Information Systems: Managing the Digital Firm*. 16th ed., Pearson, 2020.

Pressman, R. S., and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. 9th ed., McGraw-Hill Education, 2020.

Sommerville, I. *Software Engineering*. 10th ed., Pearson, 2015.

Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

Fielding, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000.

OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks*, 2021. <https://owasp.org/Top10/>

OWASP Foundation. *Application Security Verification Standard*. <https://owasp.org/www-project-application-security-verification-standard/>

Laravel, LLC. *Laravel Documentation*. <https://laravel.com/docs>

Inertia.js. *Inertia.js Documentation*. <https://inertiajs.com/>

Vue.js. *Vue.js Guide*. <https://vuejs.org/guide/>

Spatie. *Laravel-Permission Documentation*. <https://spatie.be/docs/laravel-permission/>

Standards and Further Reading

European Union. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

National Institute of Standards and Technology. Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5, 2020. <https://doi.org/10.6028/NIST.SP.800-53r5>

National Institute of Standards and Technology. Digital Identity Guidelines: Authentication and Authenticator Management, SP 800-63B, 2017, updates as published. <https://doi.org/10.6028/NIST.SP.800-63b>

W3C. Web Content Accessibility Guidelines (WCAG) 2.2, Recommendation, 2023. <https://www.w3.org/TR/WCAG22/>

PHP Group. PHP Manual. <https://www.php.net/manual/en/>

Tailwind Labs. Tailwind CSS Documentation. <https://tailwindcss.com/docs>

Apply the citation style required by the Department; verify bibliographic details and URLs before submission.

Verification Checklist

Tenant checks: create two companies; confirm each admin sees only own records; attempt foreign-tenant read, update, approval, termination, and file access; verify only super admins can switch company; confirm exit clears context.

Candidate checks: open a valid company URL; confirm only active positions; submit invalid and duplicate data; complete both assessments; test refresh and session expiry; upload supported, unsupported, empty, and oversized files; confirm consent is validated and persisted or remove the unsupported claim.

Administrative checks: validate position and questions; enforce canonical status transitions; approve repeatedly and verify one account; confirm no default password disclosure; test deactivation and retention.

Verification Execution Record

Application commit/version: _____

Operating system and PHP version: _____

Database engine and version: _____

Test command and execution date: _____

Total tests / passed / failed / skipped: _____

Environment and test database: _____

Known failures and issue references: _____

Reviewer: _____

Store test output privately. Remove personal data, tokens, environment variables, and passwords before sharing logs. A screenshot is not a substitute for assertions about database state, tenant ownership, or file access.

Use synthetic candidate records and fake storage. For browser acceptance, record role, tenant, test data, browser, expected and actual results, and any issue opened.

Complete only after tests actually run in the identified environment.

Schema and Route Reference

Primary tables: companies, users, positions, questions, candidates, candidate_assessments, plus roles, permissions, and assignment pivots. Candidate assessments link to candidates; candidates and positions are company-associated through company_id or copied position title.

Route groups: authentication and profile; company management and tenant context; protected applications, status, approval, positions, questions, and employees; public company registration, IQ, departmental assessment, documents, rules, and success.

The application uses browser web routes and Laravel sessions, not a documented stateless JSON API. Generate an exact release route inventory using `php artisan route:list`.

Suggested Validation Rules

Company names need bounded length and normalized route handling; email needs valid format and intended uniqueness. Candidate email should be normalized with explicit tenant-scoped duplicate behavior. Position must reference an active position owned by the resolved company.

Assessment answers should be arrays keyed by questions available to the candidate and follow question-type shapes. Question type and candidate status need allowlists and permitted transitions. Files need explicit requiredness, size, extension, detected MIME, storage, and retention rules.

If agreement is required, validate the Boolean, record the terms version and acceptance timestamp, and associate it with the candidate. All rules belong on the server even when the browser gives immediate feedback.

Company Administrator Guide

Sign in and confirm the company workspace. Create active positions before distributing the application link. Review question sets and test the public route in a private browser session. Review new candidate contact information, answers, and documents.

Use status actions consistently and record rationale until an audit feature exists. Approve only after eligibility and identity review. Use a secure employee onboarding link; never share a predictable password. Deactivate access using an approved procedure and follow record retention policy.

Company creation and tenant switching are privileged actions. Confirm the tenant before administration, exit the context when finished, and do not share privileged accounts. This guide does not imply audit logging or automated invitations already exist.

Candidate Application Guide

Open the organization's verified application link and confirm the company name. Enter name, email, telephone, and position. Complete assessment questions and review responses before submitting. Upload only requested valid documents and avoid shared devices where possible.

Read privacy information before indicating agreement. Contact the organization if a session expires or a document upload fails. Submit identity documents only through a verified link and ask how they are used, who can access them, how long they are retained, and how correction or deletion requests are handled.

The application should not expose another candidate's status or files. Report any unrelated personal information shown in the application.

Final Academic Submission Checklist

Confirm approved title, student name, registration, session, degree, department, college, and university affiliation. Ask the supervisor and department head to verify names and institutional details. Complete approval date and examiner fields only through the official process.

Update contents and page numbers after revision. Check citations against departmental style. Verify every implementation claim against final source and deployment. Run tests and record actual results. Proofread spelling, diagrams, page breaks, and accessibility.

This generated thesis is a detailed report draft based on the available repository. It is not an institutional approval or supervisor certification. Review the report with the supervisor before final submission.

END OF REPORT